CHMLTECH ONLINE JOURNAL · VOLUME 105, ISSUE 8 · RESEARCH ARTICLE

The Economics of Agentic AI: Cost per Task, Total Cost of Ownership and Payback

Quak Foo Lee

Chief Technology Officer, Chmltech Ltd.

Division of Artificial Intelligence and Machine Learning · Vancouver, BC, Canada

Published November 13, 2026 14 min read

ABSTRACT

The price of language model inference at a fixed level of capability has fallen by more than two orders of magnitude since 2022, yet analysts expect a large share of agentic AI projects to be cancelled because of cost and unclear value. This paper explains the gap. An agent does not pay for a prompt; it pays for a task, and a task can consume tens of thousands of tokens because each step re-reads the growing context. The paper develops a cost-per-task model in which input tokens grow quadratically with the number of steps, shows how prompt caching and batch processing change the result, and extends the model to a cost per accepted unit of work that includes human review and rework. It then sets out the full cost of ownership, including integration, evaluation, oversight and maintenance, and reviews experimental evidence on the value side, where measured productivity gains are real but uneven and depend on the task. A worked example for an accounts payable agent processing 2,000 invoices a month gives a cost per accepted invoice of about $0.63 against $4.00 by hand and a payback of about 9 months, and a sensitivity analysis shows that volume, manual handling time and build cost matter far more than token prices. The paper closes with practical rules for choosing models and use cases on economic grounds.

Keywords: agentic AI; AI economics; cost per task; total cost of ownership; inference cost; prompt caching; return on investment; productivity

1. Introduction

By one measure, artificial intelligence has become very cheap. The 2025 AI Index reports that the cost of querying a model that scores at the level of GPT-3.5 on the MMLU benchmark fell from $20 per million tokens in November 2022 to $0.07 per million tokens by October 2024, a reduction of more than 280 times in about 18 months [1]. Epoch AI, tracking the price of reaching fixed performance thresholds on several benchmarks, found declines ranging from 9 to 900 times per year depending on the task [2].

By another measure, AI agents are proving expensive. In June 2025 Gartner predicted that over 40 percent of agentic AI projects would be cancelled by the end of 2027, citing escalating costs, unclear business value and inadequate risk controls, and observed that many use cases presented as agentic do not need an agentic implementation at all [3].

Both statements can be true because they measure different things. The first measures the price of a token at a fixed level of capability. The second reflects the cost of completing a business task with an agent, including everything around the model. This paper connects the two. Section 2 builds a cost-per-task model for an agent. Section 3 extends it to the cost of an accepted unit of work, including review and rework. Section 4 sets out the full cost of ownership. Section 5 reviews the evidence on value. Section 6 combines them in a payback calculation with a sensitivity analysis, and Section 7 draws practical rules.

2. Cost per task

2.1 Why an agent uses so many tokens

A chat completion is a single call: a prompt goes in, an answer comes out. An agent works in a loop. At each step it reads its instructions, the tool definitions, the task and everything that has happened so far, decides on an action, calls a tool and appends the result to its context. Because most model APIs are stateless, each step re-sends the accumulated context as input. The cost of a task is therefore the sum over its steps:

$$C_{task} = \Sigma_k \left(n_{in,k} \cdot p_{in} + n_{out,k} \cdot p_{out} \right) + C_{tools} \quad (1)$$

where $n_{in,k}$ and $n_{out,k}$ are the input and output tokens at step k, p_{in} and p_{out} the prices per token, and C_{tools} the cost of external services such as search or document recognition.

If the fixed part of the context (system prompt, tool definitions and task) is c_0 tokens and each step adds h tokens of history, the input at step k is $c_0 + h(k - 1)$, and the total input over s steps is

$$N_{in} = s \cdot c_0 + h \cdot s(s - 1) / 2 \quad (2)$$

The second term grows with the square of the number of steps. An agent that takes twice as many steps reads roughly four times as much history. This is the main reason why the cost of an agentic task can be orders of magnitude higher than the cost of a single prompt, even when the price per token is low.

2.2 A worked example

Consider an agent that processes a supplier invoice: it reads the document, looks up the purchase order and the receipt in the ERP system, checks prices and quantities, codes the invoice and prepares a posting. Table 1 sets out illustrative parameters. Prices of $3.00 per million input tokens and $15.00 per million output tokens are typical of a capable mid-tier model in 2026; they are assumptions, not a quotation for any provider.

Table 1. Cost per task for an illustrative invoice-processing agent, from Eq. (1) and Eq. (2).

Parameter	Value
Fixed context c_0 (instructions, tool definitions, task)	4,000 tokens
History added per step h	1,500 tokens
Steps per task s	8
Output per step	400 tokens
Total input tokens N_{in}, Eq. (2)	74,000
Total output tokens	3,200
Input price / output price	$3.00 / $15.00 per million tokens
Tool calls (document recognition, search)	$0.01
Cost per task, Eq. (1)	**$0.280**
Cost per task with prompt caching	**$0.130**

Prices and token counts are illustrative assumptions.

At $0.28 per invoice the run cost is small compared with the cost of a person doing the same work. The example nonetheless shows where the money goes: input tokens account for about 79 percent of the cost, almost all of it from re-reading context. Figure 1 shows how the cost grows with the number of steps.

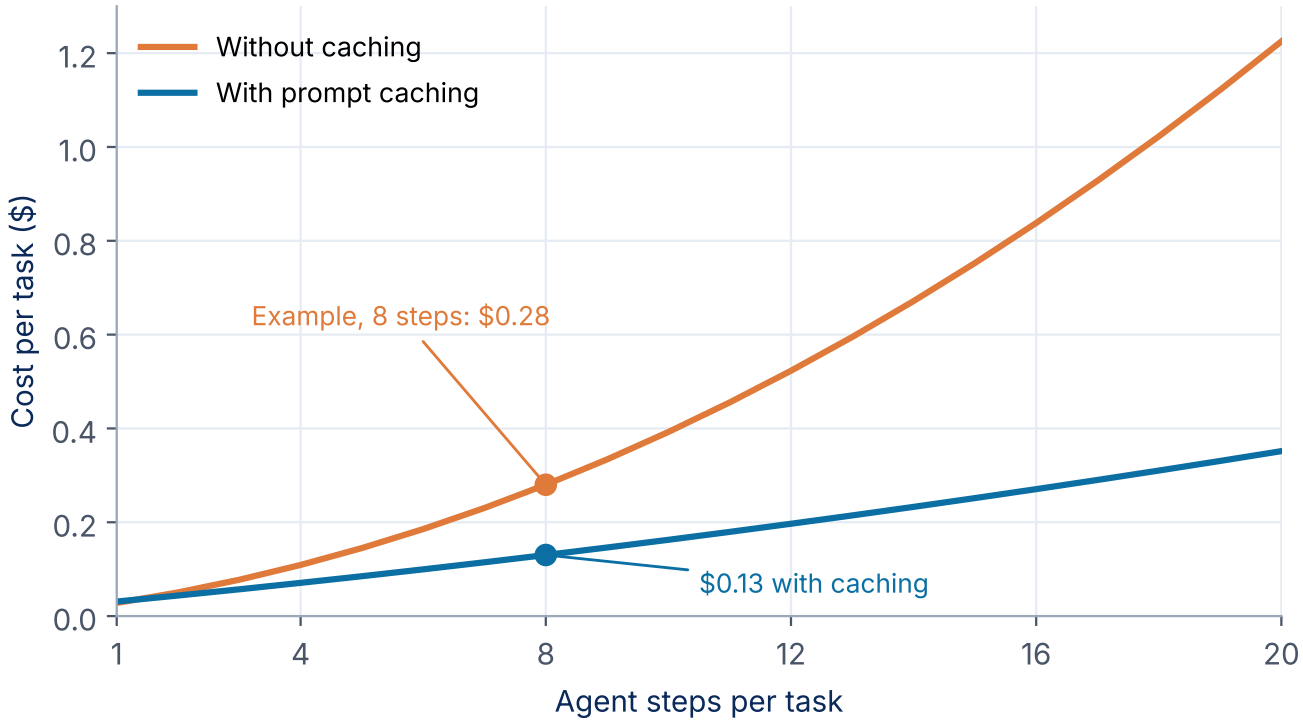


Figure 1. Cost per task against the number of agent steps, from Eq. (1) and Eq. (2), with and without prompt caching. Without caching the cost grows with the square of the number of steps because each step re-reads the history. Parameters as in Table 1.

Author's calculation from Eq. (1) and Eq. (2) with the parameters in Table 1.

2.3 Three levers on run cost

Caching. Several providers allow the stable prefix of a prompt to be cached so that repeated reads are charged at a reduced rate. In Anthropic's published pricing, for example, reading from the cache costs 10 percent of the base input price, while writing to the cache costs somewhat more than a normal input token [4]. Because an agent's context is mostly a stable prefix plus a growing history, caching attacks exactly the quadratic term in Eq. (2). In the example it reduces the cost per task from $0.280 to about $0.130.

Batching. Work that does not need an immediate answer, such as overnight document processing, can be sent through batch interfaces. OpenAI's Batch API, for example, offers a 50 percent discount in exchange for results within 24 hours [5].

Model choice. The price difference between models of different sizes is often an order of magnitude or more. Kapoor and colleagues showed that agent evaluations that ignore cost can mislead, because simple, inexpensive designs often reach similar accuracy to complex ones at a fraction of the cost, and they argued that accuracy and cost should be reported together as a

Pareto frontier [6]. A companion paper in this journal found the same pattern across frontier models: higher reasoning effort raises scores but with steeply diminishing returns per dollar [7]. Reasoning models also generate more output tokens per task, so a lower price per token does not always mean a lower price per task.

2.4 Measuring rather than estimating

Eq. (2) is a planning tool. In practice the number of steps varies from task to task: a clean invoice that matches its purchase order may take five steps, while one with a missing receipt and a price discrepancy may take fifteen, with several tool calls that return long documents. Because of the quadratic term, the expensive tail matters. A pilot should therefore log, for every task, the number of steps, the input and output tokens of each call, the tool calls made and the outcome. The distribution of cost per task, not only its average, then shows where design effort pays. Common findings are that a small share of difficult tasks consumes a large share of the tokens, that some tool calls return far more text than the agent needs, and that the agent sometimes repeats a lookup it has already made. Each of these has a direct fix: route difficult cases to a person early, trim tool outputs to the fields the agent uses, and give the agent a memory of what it has already retrieved. These fixes often reduce cost more than a change of model, and they also tend to improve accuracy, because a shorter, cleaner context is easier for the model to use.

3. Cost per accepted unit of work

The run cost is only part of the cost of a completed task. Some of the agent's outputs are reviewed by a person, and some are wrong and must be redone. The cost that matters is the cost per accepted unit of work:

$$C_{\mathrm{unit}} = C_{\mathrm{task}} + r \cdot t_{\mathrm{rev}} \cdot w + (1 - a) \cdot c_{\mathrm{redo}} \tag{3}$$

where r is the share of tasks reviewed, t_{rev} the review time, w the loaded hourly cost of the reviewer, a the share of tasks accepted, and c_{redo} the cost of doing a rejected task by hand. The comparison is with the cost of the task done entirely by a person, $c_{\mathrm{h}} = t_{\mathrm{h}} \cdot w$.

Continuing the example, suppose a clerk takes 6 minutes per invoice at a loaded cost of $40 per hour, so c_{h} = $4.00. The agent's output is reviewed for 30 percent of invoices at 1.5 minutes

each, adding $0.30 per invoice on average, and 5 percent are redone by hand, adding $0.20. With caching, Eq. (3) gives $C_{\text{unit}} \approx \$0.63$. Review and rework account for 79 percent of the cost of an accepted invoice; the model accounts for the rest.

> For most business agents, the largest cost per task is not the model but the people around it: review, rework and exception handling. Reducing review effort through better evidence and calibrated autonomy usually saves more than switching to a cheaper model.

This has two practical implications. First, the review share r is a design choice. A companion paper shows how to set it per action using the expected cost of errors and how to reduce it as evidence accumulates [8]. Second, the acceptance rate a depends on consistency as well as average accuracy. Yao and colleagues found that agents which succeed on a task once often fail on repeated attempts at the same task [9]; in production, every inconsistency becomes rework.

4. Total cost of ownership

Eq. (3) gives the marginal cost of one unit of work. A business case also needs the fixed costs of building and running the agent. Table 2 lists the main components.

Table 2. Components of the total cost of ownership of a business agent.

Component	What it covers	Often underestimated because
Integration	Connecting the agent to the ERP, CRM, document store and identity system; API work; permissions	Legacy systems lack clean interfaces; Gartner cites integration as a cause of cancellation [3]
Data preparation	Cleaning master data, documenting procedures, building a knowledge base	The agent is only as good as the data it reads [10]
Evaluation	Building a test set of real cases, defining what correct means, running it on every change	It is ongoing, not a one-time task
Inference and tools	Model calls, search, document recognition, hosting	Volumes grow once the agent works
Oversight	Review time, exception handling, sampling	Treated as temporary but often permanent at some level
Maintenance	Model version changes, prompt and tool updates, regression testing, monitoring	Model providers retire versions on their own schedule
Risk	Incidents, rework of errors that reached customers or suppliers	Rare but costly; hard to estimate in advance

Compiled by the author.

Two components deserve emphasis. Evaluation is a fixed asset: a set of a few hundred real cases with known correct outcomes, rerun whenever the model, the prompt or the tools change. Without it, a business cannot safely change models to take advantage of falling prices. Maintenance is recurring: model providers deprecate versions, and each upgrade requires a regression test. Both costs are modest compared with integration but are rarely in the original budget.

For the invoice example, the build cost assumed in Section 6 can be broken down as follows: about half for integration with the ERP system and the document store, including read and write permissions and a test environment; about a quarter for data preparation, mainly cleaning supplier and item master data and writing down the coding rules that experienced clerks apply

from memory; and the remainder for an evaluation set of several hundred past invoices with their correct coding, plus the pilot itself. The fixed monthly cost covers hosting, monitoring dashboards, a share of an analyst's time to review exceptions and metrics, and periodic regression testing. None of these items depends much on the price of the model.

5. The value side

5.1 What experiments show

The best evidence on the value of generative AI at work comes from controlled studies. Brynjolfsson, Li and Raymond studied the staggered introduction of an AI assistant for over 5,000 customer support agents and found that it increased issues resolved per hour by 14 percent on average, and by 34 percent for novice and low-skilled workers, with little effect on the most experienced [11]. Noy and Zhang found in an online experiment with professional writing tasks that access to ChatGPT reduced the time taken by about 40 percent and raised output quality by about 18 percent [12]. Dell'Acqua and colleagues, in a field experiment with consultants, found that for tasks inside the AI's capability frontier, consultants with AI completed 12.2 percent more tasks, 25.1 percent more quickly and with more than 40 percent higher quality; for a task outside the frontier, those using AI were 19 percentage points less likely to reach a correct answer [13].

5.2 What that means for a business case

These studies support three cautions. Gains are concentrated in tasks well within the model's capabilities, and can turn negative outside them, which Dell'Acqua and colleagues called a jagged frontier [13]. Gains are larger for less experienced staff, so the benefit depends on who does the work today. And measured gains are per task; at the level of the economy, Acemoglu estimates that the total factor productivity effect of AI over ten years is likely to be modest, in part because the share of tasks that can be profitably automated is limited [14].

For a single business the most important point is that time saved is not money saved unless the time is used. If an agent saves a clerk three hours a day, the value is realized only if the business processes more volume without hiring, redeploys the clerk to work it needs done, or reduces overtime or contractor spend. A business case should state which of these will happen.

5.3 Value beyond labour cost

Labour time is the easiest benefit to measure but often not the largest. An accounts payable agent that posts invoices within a day rather than a week lets a business take early-payment discounts and avoid late fees, and gives a more current view of liabilities. A maintenance agent that drafts work orders from inspection notes shortens the time between finding a problem and fixing it. An agent that answers routine supplier or customer questions outside business hours improves response times without adding a shift. Agents can also reduce error rates where the human process is itself error-prone, for example in transcribing figures from documents. These benefits should be included in a business case where they can be measured, but each needs a baseline taken before the agent is introduced. Without a baseline, a business cannot show the benefit afterwards, and the case rests on the labour saving alone.

6. Payback

6.1 A simple payback model

With the cost per unit from Eq. (3), a monthly volume V, fixed monthly costs F for hosting, monitoring and maintenance, and a one-time build cost B, the payback period in months, Eq. (4), is

$$T_{\text{payback}} = B \,/\, \left(V \cdot (c_h - C_{\text{unit}}) - F \right) \tag{4}$$

The denominator must be positive: if the savings per unit times the volume do not cover the fixed monthly costs, the agent never pays back.

For the invoice example, with V = 2,000 invoices a month, savings of $3.37 per invoice give gross savings of $6,740 a month. Fixed costs of $1,500 a month leave $5,240. A build cost of $45,000 for integration, data preparation and an evaluation set then pays back in about 8.6 months.

6.2 What the result is sensitive to

Figure 2 varies each input by a plausible range and shows the effect on the payback period. Volume dominates: at 1,000 invoices a month the payback period stretches to about two years, because the fixed costs are spread over fewer invoices. The time the work takes by hand and

the build cost come next. Review share and rework matter less in this example because the agent's cost per invoice is already small relative to the human cost, though raising the review share from 30 to 60 percent still adds about a month. Token prices barely matter at this scale: doubling the price of the model changes the payback by a few days.

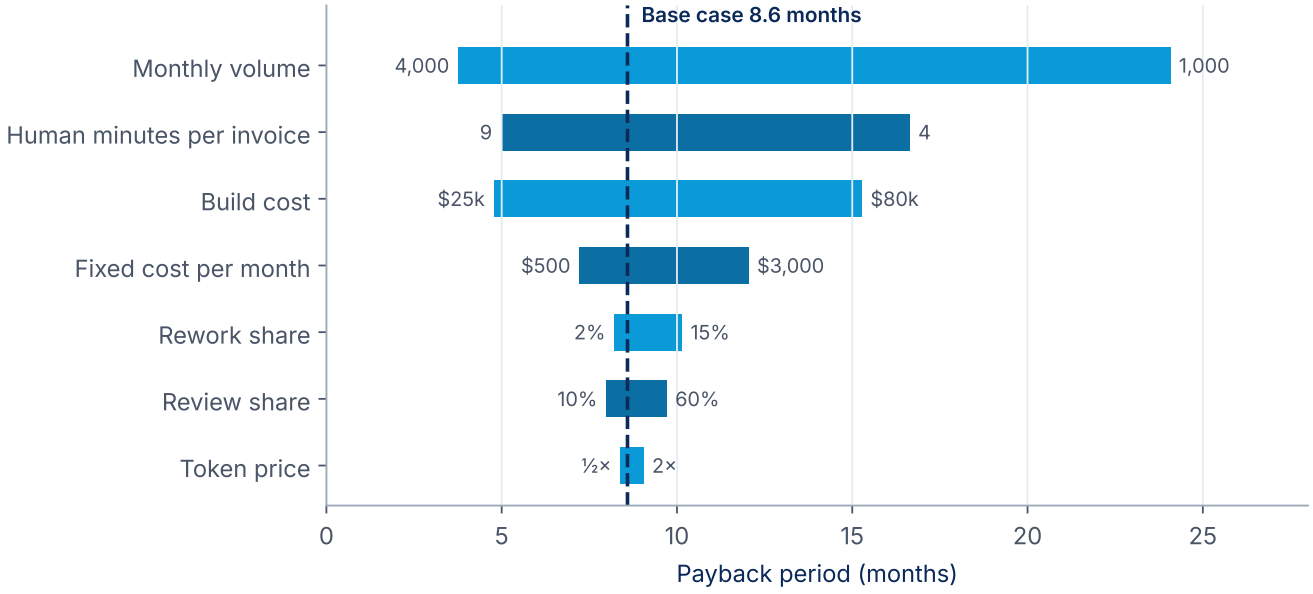


Figure 2. Sensitivity of the payback period for the invoice example to each input, varied one at a time over the range shown. Volume, manual handling time and build cost dominate; the model's token price has little effect.

Author's calculation from Eq. (3) and Eq. (4) with the parameters in Section 6.1.

A related figure is the break-even volume: the monthly volume at which savings just cover the fixed monthly costs, so that the denominator of Eq. (4) is zero. It equals $F / (c_h - C_{unit})$, or about 445 invoices a month in the example. Below that volume the agent loses money every month whatever its build cost. The break-even volume is a quick screening test for candidate processes: a business that handles a few dozen of a given document a month is unlikely to justify a dedicated agent for it, though it may still benefit from a general assistant that its staff use for many small tasks.

This is consistent with the observation that projects fail on cost and value rather than on token prices [3]. A low-volume process will not pay back however cheap the model becomes, while a high-volume, well-structured process can pay back quickly even with an expensive model.

7. Practical rules

The analysis suggests six rules for deciding where agents make economic sense.

1. **Price the task, not the token.** Estimate steps, context and output per task with Eq. (1) and Eq. (2), and measure them in a pilot. Count reasoning tokens.
2. **Keep loops short and contexts lean.** Each extra step is paid for many times over. Summarize history, pass only the documents needed, and cache stable prefixes.
3. **Start with high-volume, rule-based work.** Volume spreads the fixed costs; clear rules keep review and rework low. Low-volume, judgment-heavy tasks are poor first candidates.
4. **Budget for the people.** Review, exceptions and rework are usually the largest variable cost. Design for evidence-based reduction of review.
5. **Build the evaluation set first.** It is what allows a later switch to a cheaper or better model, which is how a business captures the price decline over time [2, 6].
6. **Say where the saved time goes.** A business case without a plan for the freed capacity counts savings that will not appear in the accounts.

7.1 When not to use an agent

Gartner's observation that many use cases presented as agentic do not need an agentic implementation [3] has a simple economic basis. If a task follows fixed rules, has structured inputs and rarely varies, conventional automation (a script, a workflow rule or robotic process automation) does it at near-zero marginal cost and with no review. An agent adds value where inputs are unstructured, such as free-text emails and scanned documents, where the right action depends on judgment within known limits, or where the variety of cases is too large to write rules for. A useful test is to ask whether a competent person could write down the complete rule set in a day. If so, write the rules. If the rules would run to hundreds of exceptions, an agent may be the cheaper option, with the rules it does follow written into its instructions and tools.

8. Conclusion

The falling price of tokens is real, but it is not the price of a task. An agent pays for every step it takes and re-reads its growing context each time, so its run cost grows with the square of the number of steps unless caching is used. The run cost, however, is usually not the main cost. Review, rework, integration, evaluation and maintenance dominate, and the worked example shows that the payback of a typical business agent depends far more on volume and review

effort than on the model's price. The evidence on value is encouraging for tasks within the model's capabilities and cautionary outside them. A business that prices tasks rather than tokens, starts with high-volume structured work, invests in evaluation and plans the use of freed time is well placed to be among the projects that continue.

References

1. Stanford Institute for Human-Centered Artificial Intelligence. (2025). *The 2025 AI Index report*. Stanford University. hai.stanford.edu/ai-index/2025-ai-index-report
2. Epoch AI. (2025, March). LLM inference prices have fallen rapidly but unequally across tasks [Data insight]. epoch.ai/blog/llm-inference-price-trends
3. Gartner. (2025, June 25). Gartner predicts over 40% of agentic AI projects will be canceled by end of 2027 [Press release]. www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-p
4. Anthropic. (2026). Prompt caching [Developer documentation]. Retrieved November 13, 2026. docs.claude.com/en/docs/build-with-claude/prompt-caching
5. OpenAI. (2026). Batch API [Developer documentation]. Retrieved November 13, 2026. platform.openai.com/docs/guides/batch
6. Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2024). AI agents that matter. arXiv:2407.01502. arxiv.org/abs/2407.01502
7. Lee, Q. F. (2026). Score versus cost per task: Measuring the returns to reasoning effort in frontier language models. *Chmltech Online Journal*, 104(7). www.chmltech.com/post/score-versus-cost-per-task-returns-to-reasoning-effort-in-
8. Lee, Q. F. (2026). Levels of autonomy and human-in-the-loop design for AI agents. *Chmltech Online Journal*, 105(7). www.chmltech.com/post/levels-of-autonomy-and-human-in-the-loop-design-for-ai-age
9. Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045. arxiv.org/abs/2406.12045
10. Lee, Q. F. (2026). Agent readiness for small and mid-sized firms: Data foundations, process documentation and staged adoption. *Chmltech Online Journal*, 105(6). www.chmltech.com/post/agent-readiness-for-small-and-mid-sized-firms-data-process
11. Brynjolfsson, E., Li, D., & Raymond, L. (2025). Generative AI at work. *The Quarterly Journal of Economics*, 140(2), 889–942. 10.1093/qje/qjae044
12. Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654), 187–192. 10.1126/science.adh2586

13. Dell'Acqua, F., McFowland, E., Mollick, E. R., Lifshitz-Assaf, H., Kellogg, K., Rajendran, S., Krayer, L., Candelon, F., & Lakhani, K. R. (2023). Navigating the jagged technological frontier: Field experimental evidence of the effects of AI on knowledge worker productivity and quality (Harvard Business School Working Paper 24-013). 10.2139/ssrn.4573321
14. Acemoglu, D. (2025). The simple macroeconomics of AI. *Economic Policy*, 40(121), 13–58. 10.1093/epolic/eiae042

How to cite this article

Lee, Q. F. (2026). The Economics of Agentic AI: Cost per Task, Total Cost of Ownership and Payback. *Chmltech Online Journal*, *105*(8). https://www.chmltech.com/post/the-economics-of-agentic-ai-cost-per-task-total-cost-and-payback